

ASSIGNMENT 13

Name: Rajdeep Das

Roll No: CSB25203

Subject: Advanced Programming

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
/* Structure for dynamic string buffer */
```

```
typedef struct
```

```
{
```

```
    char    *data;
```

```
    size_t   length;
```

```
    size_t capacity;
```

```
} StringBuffer;
```

```
/* Initialize the string buffer */
```

```
StringBuffer *sb_init(size_t initial_capacity)
```

```
{
```

```
    StringBuffer *sb;
```

```
    sb = (StringBuffer *)malloc(sizeof(StringBuffer));
```

```
    if (sb == NULL)
```

```
    {
```

```
        printf("Failed to allocate StringBuffer\n");
```

```
    return NULL;
```

```
    }
```

```

    sb->data = (char *)malloc(initial_capacity);
if (sb->data == NULL)
{
    printf("Failed to allocate buffer\n");
    free(sb);
return NULL;
}

```

```

    sb->length = 0;    sb->capacity
= initial_capacity;

```

```

    sb->data[0] = '\0';

```

```

    return sb;
}

```

```

/* Append string to buffer */ int
sb_append(StringBuffer *sb, const char *str)
{
    size_t    str_len;
size_t    required_size;
char *temp;

```

```

    if (sb == NULL || str == NULL)
    {
        return 0;
    }

```

```

    str_len = strlen(str);

```

```

    /* +1 for null terminator */
    required_size = sb->length + str_len + 1;

    /* Grow buffer if needed */ while
    (required_size > sb->capacity)
    {
        size_t new_capacity = sb->capacity * 2;

        temp = (char *)realloc(sb->data, new_capacity);

        /* Safe realloc handling */
        if (temp == NULL)
        {
            printf("Realloc failed\n");
            return 0;
        }

        sb->data = temp;    sb-
        >capacity = new_capacity;

        printf("Buffer grown to capacity: %lu\n",
            (unsigned long)sb->capacity);
    }

    /* Append new string */ strcpy(sb-
    >data + sb->length, str);

    sb->length += str_len;

    return 1;
}

```

```

/* Free all memory */ void
sb_free(StringBuffer *sb)
{   if (sb !=
    NULL)
    {
        free(sb->data);
    free(sb);
    }
}

int main()
{
    StringBuffer *sb;

    sb = sb_init(8);

    if (sb == NULL)
    {
        return 1;
    }

    printf("Initial capacity: %lu\n",
        (unsigned long)sb->capacity);

    sb_append(sb, "Hello");
    printf("Buffer: %s\n", sb->data);

    sb_append(sb, ", World!"); printf("Buffer:
    %s\n", sb->data);   sb_append(sb, " This

```

```
is a dynamic string buffer example.");  
printf("Buffer: %s\n", sb->data);  
  
printf("\nFinal String: %s\n", sb->data);  
printf("Final Length: %lu\n",  
(unsigned long)sb->length);  
printf("Final Capacity: %lu\n",  
      (unsigned long)sb->capacity);  
  
/* Free memory */  
sb_free(sb);  
  
return 0;  
}
```